

Evaluation + Optimization

CS-3180

So far

- Two big things we've talked about:
- Execution methods
(compilers/interpreters)
- Type Systems
- At least the deep dives



Core Difference

- The "scope" of what executors see
- Including how they see your types
- Evaluate differently:
- Different capabilities of evaluation
- Specifically at different times

Interpreters

- Does EVERYTHING at runtime
- This usually extends to executing new code
- "At runtime, evaluate this new piece of code"
- Most interpreted languages have a nice function for this
- Python's is `eval()`

eval()

- "Evaluate a piece of code"
- Accepts an expression
- Executes and evaluates that code
- Returns its result

```
result = eval("5 * 4 + 2")  
print(result)
```

Eval()

- This is cool
- Almost like a separate runtime
- It can even access surrounding values

```
x = 10
y = 20
result = eval("x + y")
print(result)
```

EvilO

- This is very scary
- Huge security risk
- Think about what we are doing
- Execute *arbitrary code*?
- Let's see that



- We can call destructive functions
- Rule: Don't use this on un-trusted input

```
def evil_fn():  
    return "rm -rf /"  
  
def main():  
    to_eval = input("Give me something to evaluate: ")  
    result = eval(to_eval)  
    print(result)  
  
if __name__ == "__main__":  
    main()
```

Eval + Interpreters

- This is *mostly* unique to interpreted languages
- Although not entirely
- How would a compiled language even handle this?
- Compiled languages often produce binaries
- Those binaries are fixed, as is

EvalO

```
int main() {  
    system("gcc -o exec_me file.c");  
    system("./exec_me");  
    return 0;  
}
```

- Fake C Eval attempt
- This doesn't address access to existing values
- Point being: Fundamentally different

Compilers

- If compilers can't easily do dynamic evaluation
- What other advantages do they have?
- Since they can "see" more of your code
- They can do additional optimizations



Compiler Optimizations

- Compiler's will change you code
- Often much much more than you think
- "As-If rule"
- Your code should execute as-if it was not changed



Compiler Optimizations

- Every single line of high-level code
- Produces several lines of low-level code
- Different instructions take different time
- Most of the time
- However, more instructions = more time

C to ASM

```
int i;  
for (i = 0; i < 10; i++) {  
    // some code here  
}
```

```
mov ecx, 0           ; initialize loop  
.loop_start:         ; label  
cmp ecx, 10          ; conditional  
jge .loop_end        ; exit loop  
; some code here  
inc ecx              ; increment var  
jmp .loop_start      ; jump back  
.loop_end:           ; label
```

Assembly

- Every new assembly instruction takes time
- Would like to operate on this representation
- (More commonly, an AST)
- To minimize these instructions
- Assembly is *inherently* imperative
- Let's look at some different optimizations

Dead Code Elimination

- Let's start small
- If code can not run
- Don't load the assembly
- Compiler must **PROVE** it can't run
- Java won't even compile

```
int foo() {  
    int a = 5;  
    int b = 5;  
  
    return a + b;  
  
    if (a == 5) {  
        return 0;  
    }  
}
```

Conditional Dead Code

- Can do the same with conditional

```
1 class Optimize {
2     public static void main(String[] args) {
3         System.out.println(foo());
4     }
5
6     public static int foo(){
7         int a = 5;
8         int b = 5;
9
10        if(false){
11            return 0;
12        }
13
14        return a + b;
15    }
16 }
```

Conditional Dead Code

- Can do the same with conditional

```
1 class Optimize {
2     public static void main(String[] args) {
3         System.out.println(foo());
4     }
5
6     public static int foo(){
7         int a = 5;
8         int b = 5;
9
10        if(false){
11            return 0;
12        }
13
14        return a + b;
15    }
16 }
```

Conditional Dead Code

- Can do the same with conditional

```
1 class Optimize {
2     public static void main(String[] args) {
3         System.out.println(foo());
4     }
5
6     public static int foo(){
7         int a = 5;
8         int b = 5;
9
10        if(false){
11            return 0;
12        }
13
14        return a + b;
15    }
16 }
```

Conditional Dead Code

- Delete that portion of code

```
class Optimize {  
    public static void main(String[] args) {  
        System.out.println(foo());  
    }  
  
    public static int foo(){  
        int a = 5;  
        int b = 5;  
  
        return a + b;  
    }  
}
```

Conditional Optimization

- Can optimize conditionals
- If compiler can prove a case

```
if(false && conditional){  
    // will never run  
    // Drop entire conditional  
}
```

```
if(true || conditional){  
    // code will ALWAYS run  
    // take code out of a condition  
    // less ASM to load  
}
```

Short Circuit Evaluation

- If only one conditional is required
- To determine whether a block runs
- Only that call gets executed

```
if (foo() || bar()){  
    // some code  
    // if foo() return true  
    // bar never runs  
}
```

Const Folding

- Unchanging values (final) only need computed one time
- Evaluate it at COMPILE time
- Instead of loading those into assembly

```
int seconds = 60 * 60 * 24;  
wait_for(seconds)
```

```
int seconds = 86400;  
wait_for(seconds)
```

Const Propagation

- Variable access costs instructions
- Substitute a variable for a value
- If it is known
- Chain with constant folding

```
1 int x = 14;  
2 int y = 7 - (x / 2);  
3 return y * 3;
```

```
// replace every instance of x with 14  
// fold, then replace every instance of y with 7  
return 7 * 3  
// would optimize this to:  
return 21
```

Const Propagation

- Variable access costs instructions
- Substitute a variable for a value
- If it is known
- Chain with constant folding

```
1 int x = 14;  
2 int y = 7 - (x / 2);  
3 return y * 3;
```

```
// replace every instance of x with 14  
// fold, then replace every instance of y with 7  
return 7 * 3  
// would optimize this to:  
return 21
```

Const Propagation

- Variable access costs instructions
- Substitute a variable for a value
- If it is known
- Chain with constant folding

```
1 int x = 14;  
2 int y = 7 - (x / 2);  
3 return y * 3;
```

```
// replace every instance of x with 14  
// fold, then replace every instance of y with 7  
return 7 * 3  
// would optimize this to:  
return 21
```

Const Propagation

- Variable access costs instructions
- Substitute a variable for a value
- If it is known
- Chain with constant folding

```
1 int x = 14;  
2 int y = 7 - (x / 2);  
3 return y * 3;
```

```
// replace every instance of x with 14  
// fold, then replace every instance of y with 7  
return 7 * 3  
// would optimize this to:  
return 21
```

Const Propagation

- Variable access costs instructions
- Substitute a variable for a value
- If it is known
- Chain with constant folding

```
1 int x = 14;  
2 int y = 7 - (x / 2);  
3 return y * 3;
```

```
// replace every instance of x with 14  
// fold, then replace every instance of y with 7  
return 7 * 3  
// would optimize this to:  
return 21
```

Side Effects

- Many of these optimizations rely on the simple usage
- Functions don't do anything weird
- This is worth spending some time on
- It will become relevant later

Side Effects

- Call a side-effect:
- A state modification outside of the functions scope

```
balance = 100

def withdraw_impure(amount):
    global balance
    balance -= amount # mutate global state
    return balance

print(withdraw_impure(30)) # 70
print(withdraw_impure(30)) # 40
print(withdraw_impure(30)) # 10
```

Pure Functions

- Call a Pure Function:
- A function that does NOT mutate external state
- Necessary for many optimizations

```
def withdraw_pure(current_balance, amount):  
    # no state mutation  
    return current_balance - amount
```

```
balance = 100
```

```
balance = withdraw_pure(balance, 30)  
print(balance) # 70
```

```
balance = withdraw_pure(balance, 30)  
print(balance) # 40
```

```
balance = withdraw_pure(balance, 30)  
print(balance) # 10
```

In-lining

- Replace pure functions calls with the code itself
- Literally copy+pasted
- Prevent loading a new function call
- C++ has the ``constexpr`` for evaluating entire functions at compile time
- AND in-lining them

constexpr

```
constexpr int square(int x) {  
    return x * x;  
}  
  
int main() {  
    // square(5) is determined entirely at COMPILE TIME  
    constexpr int static_val = square(5);  
}
```

- My favorite C++ feature
- Rust has something similar
- Zig can do WAY more

Caching

- Lookup of variables is expensive
- "Common" values show up enough
- For certain languages to "cache" them
- At least, references to them
- To prevent unnecessary lookup

```
class Cache {  
    public static void main(String[] args){  
        // System.out.println(100 == 100);  
        // System.out.println(200 == 200);  
        Integer a = 100;  
        Integer b = 100;  
  
        System.out.println(a == b); // true  
  
        Integer x = 200;  
        Integer y = 200;  
        System.out.println(x == y); // false  
    }  
}
```

Summary

- Interpreters can evaluate code dynamically
- Easier than compilers can at least
- Compilers can do many more optimizations
- They can "see" more of your code

Questions?